

LAPORAN FINAL ANALISIS ARSITEKTUR SISTEM

Sistem Informasi Akademik

Dokumen: Laporan Analisis Class Diagram UML

Versi: 2.0

Tanggal: 7 Oktober 2025

Disusun oleh: Duwi Joko AP-24364701

DAFTAR ISI

1. Pendahuluan
2. Ringkasan Eksekutif
3. Tinjauan Arsitektur Sistem
4. Analisis Detail Komponen Sistem
5. Analisis Relasi dan Kardinalitas
6. Evaluasi Prinsip SOLID
7. Verifikasi dan Validasi Diagram
8. Kesimpulan dan Rekomendasi
9. Lampiran

1. PENDAHULUAN

1.1 Tujuan Dokumen

Dokumen ini bertujuan untuk menyajikan analisis menyeluruh terhadap desain arsitektur **Sistem Informasi Akademik** berdasarkan **Class Diagram UML** yang telah dibuat. Analisis mencakup evaluasi struktur kelas, relasi antar komponen, penerapan prinsip desain perangkat lunak, serta rekomendasi untuk implementasi.

1.2 Ruang Lingkup

Sistem Informasi Akademik ini mencakup:

- Manajemen pengguna (Mahasiswa, Dosen, Admin)
- Proses Penerimaan Mahasiswa Baru (PMB)
- Pengelolaan mata kuliah dan jadwal
- Sistem Kartu Rencana Studi (KRS)
- Manajemen nilai dan perhitungan IPK
- Dokumen akademik

1.3 Metodologi Analisis

Analisis dilakukan dengan pendekatan:

- **Structural Analysis:** Evaluasi struktur kelas dan atribut
- **Behavioral Analysis:** Analisis metode dan interaksi antar objek
- **Design Pattern Review:** Identifikasi pola desain yang diterapkan
- **SOLID Principles Verification:** Validasi penerapan prinsip-prinsip SOLID

2. RINGKASAN EKSEKUTIF

Sistem Informasi Akademik yang dianalisis menunjukkan **arsitektur yang matang dan well-designed**. Desain ini mengadopsi pendekatan *object-oriented* dengan penerapan prinsip **SOLID** yang eksplisit, pemisahan *concerns* yang jelas antara entitas domain dan *service layer*, serta struktur relasi yang logis dan terverifikasi.

Kekuatan Utama:

- ☒ Penerapan prinsip **SOLID** yang konsisten
- ☒ **Separation of Concerns** yang jelas
- ☒ Relasi antar kelas yang terverifikasi tanpa *circular dependency*
- ☒ **Service layer** yang mendukung *maintainability*
- ☒ Struktur hierarki yang *clean* dan *extensible*

Area Perhatian:

- ⚠ Kompleksitas **validasi KRS** memerlukan *testing* yang komprehensif.
 - ⚠ Perlu dokumentasi detail untuk logika **perhitungan grade**.
-

3. TINJAUAN ARSITEKTUR SISTEM

3.1 Pola Arsitektur

Sistem ini menggunakan **Layered Architecture** dengan pemisahan yang jelas:

- **Presentation Layer** (User Interface - *tidak digambar*)
- **Business Logic Layer**
 - User, Mahasiswa, Dosen, Admin
 - Course, KRS, Enrollment
 - PMB, CalonMahasiswa
- **Service Layer**
 - KRSValidationService
 - GradeCalculationService
- **Data Access Layer** (Persistence - *tidak digambar*)

3.2 Modul Fungsional Utama

Sistem terbagi menjadi 6 modul fungsional:

No	Modul	Kelas Utama	Tanggung Jawab
1	User Management	User, Mahasiswa, Dosen, Admin	Autentikasi, otorisasi, manajemen profil
2	PMB (Penerimaan)	PMB, CalonMahasiswa, PMBStatistics	Proses pendaftaran mahasiswa baru
3	Academic Core	Course, Schedule, Semester, AcademicYear	Manajemen kurikulum dan jadwal
4	Enrollment	KRS, Enrollment	Pengisian KRS dan registrasi mata kuliah

5	Grading	Grade, GradeCalculationService	Penilaian dan perhitungan IPK
6	Document	Document, DocumentType	Manajemen dokumen akademik

4. ANALISIS DETAIL KOMPONEN SISTEM

4.1 Modul User Management

4.1.1 Class: User (Abstract)

Deskripsi: Kelas abstrak sebagai base class untuk semua jenis pengguna.

Atribut Kritis: pmbld: String, tahunAkademik: String, periode: String, startDate: Date, endDate: Date, currentApplicants: int, requirements: List<String>.

Metode Utama: login(): boolean, logout(): void, changePassword(): boolean, getRole(): String.

Prinsip SOLID: ☒ SRP (Fokus pada autentikasi dan manajemen sesi), ☒ OCP (Dapat diperluas melalui inheritance).

4.1.2 Class: Mahasiswa

Deskripsi: Representasi mahasiswa dalam sistem.

Atribut Spesifik: mahasiswald: String (NIM), angkatan: String, fakultas: String, prodi: String, semester: int, ipk: double, totalSks: int.

Metode Utama: enrollCourse(course: Course): Enrollment, viewTranscript(): List<Grade>, calculateIPK(): double.

Relasi: Inherits dari User, Creates KRS (1 to 0..), Enrolled in Enrollment (1 to 0..).

4.1.3 Class: Dosen

Deskripsi: Representasi dosen pengajar.

Atribut Spesifik: nip: String, jabatan: String, departemen: String, expertise: List<String>.

Metode Utama: createCourse(): Course, assignGrade(enrollment: Enrollment, grade: Grade): void, viewStudentList(): List<Mahasiswa>, updateSchedule(): void.

Relasi: Inherits dari User, Teaches Course (1 to 0..*).

4.1.4 Class: Admin

Deskripsi: Administrator sistem dengan hak akses penuh.

Atribut Spesifik: adminId: String, accessLevel: String.

Metode Utama: getRole(): String, systemConfiguration(): void.

4.2 Modul PMB (Penerimaan Mahasiswa Baru)

4.2.1 Class: PMB

Deskripsi: Mengelola proses penerimaan mahasiswa baru.

Atribut: pmbId: String, tahunAkademik: String, periode: String, startDate: Date, endDate: Date, currentApplicants: int, requirements: List<String>.

Metode Utama: openRegistration(): void, closeRegistration(): void, processApplication(application: CalonMahasiswa): void, announceResults(): void, generateStatistics(): PMBStatistics.

Prinsip SOLID: ☒ OCP ("Extensible for different admission processes").

4.2.2 Class: CalonMahasiswa

Deskripsi: Representasi calon mahasiswa yang mendaftar.

Atribut: calonMahasiswald: String, registrationConfirmation: String, status: PMBStatus, documents: List<Document>.

Metode: submitApplication(): void, uploadDocuments(): void, checkApplicationStatus(): PMBStatus.

Relasi: Associated with PMB (many to 1), Has Document (1 to 0..*).

4.2.3 Class: PMBStatistics

Deskripsi: Agregasi statistik dari proses PMB.

Atribut: totalApplicants: int, acceptedApplicants: int, rejectedApplicants: int, pendingApplicants: int.

Metode: generateReport(): String.

4.3 Modul Academic Core

4.3.1 Class: Course

Deskripsi: Representasi mata kuliah.

Atribut: courseId: String, name: String, sks: int, description: String, semester: int, isActive: boolean, maxStudents: int, prerequisites: List<Course>.

Metode: addPrerequisite(course: Course): void, checkPrerequisite(mahasiswa: Mahasiswa): boolean, isAvailable(): boolean.

Relasi: Taught by Dosen (many to 1), Has Schedule (1 to 1..), Has prerequisite Course (self-reference, 0..* to 0..*), Enrolled through Enrollment (1 to 0..).

4.3.2 Class: Schedule

Deskripsi: Jadwal perkuliahan.

Atribut: scheduleId: String, hari: DayOfWeek, jamMulai: Time, jamSelesai: Time, kapasitas: int, semester: String, tahunAkademik: String.

Metode: isConflictedFor(schedule: Schedule): boolean, isAvailable(): boolean.

4.3.3 Class: Semester

Deskripsi: Representasi semester akademik.

Atribut: semesterId: String, name: String, academicYear: AcademicYear, startDate: Date, isActive: boolean.

4.3.4 Class: AcademicYear

Deskripsi: Tahun akademik.

Atribut: yearId: String, name: String, isActive: boolean, startDate: Date.

4.4 Modul Enrollment (KRS)

4.4.1 Class: KRS

Deskripsi: Kartu Rencana Studi mahasiswa.

Atribut: krsId: String, semester: String, tahunAkademik: String, totalSKS: int, status: KRSStatus.

Metode: addCourse(course: Course): boolean, removeCourse(course: Course): boolean, calculateTotalSKS(): int, submit(): void, approve(): void.

Relasi: Created by Mahasiswa (many to 1), Contains Enrollment (1 to 1..*), Uses KRSValidationService (dependency).

Prinsip SOLID: ☒ ISP (Validasi dipisahkan ke KRSValidationService).

4.4.2 Class: Enrollment

Deskripsi: Bridge class antara Mahasiswa dan Course melalui KRS.

Atribut: enrollmentId: String, enrollmentDate: Date, semester: String, tahunAkademik: String, status: EnrollmentStatus.

Metode: drop(): void, getStatus(): EnrollmentStatus.

Relasi: Links Mahasiswa (many to 1), Links Course (many to 1), Part of KRS (many to 1), Results in Grade (1 to 0..1).

4.4.3 Service: KRSValidationService

Deskripsi: Service untuk validasi KRS.

Metode: validatePrerequisites(mahasiswa: Mahasiswa, course: Course): boolean, validateSKSLimit(krs: KRS): boolean, validateMahasiswaStudent(course: Course): boolean.

Prinsip SOLID: ☒ ISP ("Separated validation logic from KRS entity"), ☒ SRP (Fokus hanya pada validasi).

4.5 Modul Grading

4.5.1 Class: Grade

Deskripsi: Nilai mahasiswa untuk suatu mata kuliah.

Atribut: gradeId: String, nilaiAngka: double, nilaiHuruf: String, semester: String, tahunAkademik: String.

Metode: calculateBobot(): double, isPas(): boolean, getGradePoints(): double.

Relasi: Result of Enrollment (many to 1).

4.5.2 Service: GradeCalculationService

Deskripsi: Service untuk perhitungan nilai dan IPK.

Metode: calculateSemesterGPA(grades: List<Grade>): double, calculateCumulativeGPA(mahasiswa: Mahasiswa): double, calculateBobotNilaiAngka(grade: double, sks: int): double.

Prinsip SOLID: ☒ DIP ("Distributed modules depend on this service abstraction"), ☒ SRP (Fokus pada perhitungan grade).

4.6 Modul Document

4.6.1 Class: Document

Deskripsi: Dokumen akademik yang diunggah.

Atribut: documentId: String, type: DocumentType, fileName: String, uploadDate: Date, verifiedDate: Date, isVerified: boolean.

Metode: verify(): boolean, download(): byte[].

4.6.2 Enum: DocumentType

Values: TRANSKIP, KTP, IJAZAH, FOTO, SURAT_KETERANGAN_SEHAT.

5. ANALISIS RELASI DAN KARDINALITAS

5.1 Tabel Relasi Utama

Dari	Ke	Tipe	Kardinalitas	Penjelasan
User	Mahasiswa	Inheritance	-	IS-A relationship
Mahasiswa	KRS	Association	1 to 0..*	Mahasiswa membuat banyak KRS
KRS	Enrollment	Composition	1 to 1..*	KRS berisi banyak Enrollment

Course	Schedule	Composition	1 to 1..*	Course memiliki jadwal
Enrollment	Grade	Association	1 to 0..1	Enrollment menghasilkan 1 Grade
PMB	CalonMahasiswa	Association	1 to 0..*	PMB memproses banyak pendaftar
AcademicYear	Semester	Composition	1 to 1..*	Tahun terdiri dari semester
Course	Course	Self-reference	0..* to 0..*	Prasyarat mata kuliah

5.2 Verifikasi Kardinalitas

☒ Verifikasi Passed:

- Semua kardinalitas sudah logis dan sesuai dengan domain bisnis.
- Relasi *many-to-many* (Mahasiswa-Course) sudah di-*bridge* dengan **Enrollment**.

6. EVALUASI PRINSIP SOLID

6.1 Single Responsibility Principle (SRP)

Status: ☒ TERPENUHI

Bukti: Kelas **User** fokus pada autentikasi. Kelas **Service** (`KRSValidationService`, `GradeCalculationService`) memiliki tanggung jawab tunggal yang jelas.

6.2 Open/Closed Principle (OCP)

Status: ☒ TERPENUHI

Bukti: Kelas **PMB** dirancang "*Extensible for different admission processes*". Hierarki **User** mendukung penambahan tipe *user* baru tanpa modifikasi kode *existing*.

6.3 Liskov Substitution Principle (LSP)

Status: ☒ TERPENUHI

Bukti: Semua *subclass* dari **User** dapat menggantikan *parent class* (**User**) tanpa menimbulkan kesalahan (e.g., `User user = new Mahasiswa();`).

6.4 Interface Segregation Principle (ISP)

Status: ☒ TERPENUHI

Bukti: Logika validasi dipisahkan ke **KRSValidationService**, mencegah entitas **KRS** dipaksa mengimplementasikan metode yang tidak relevan.

6.5 Dependency Inversion Principle (DIP)

Status: ☒ TERPENUHI

Bukti: Modul *high-level* (seperti `Mahasiswa.calculateIPK()`) bergantung pada abstraksi (*service layer*), bukan pada implementasi konkret dari perhitungan grade.

7. VERIFIKASI DAN VALIDASI DIAGRAM

7.1 Inheritance Verification

Status: ☒ PASSED

- Pohon *inheritance* bersih tanpa ambiguitas.
- *Abstract class* (**User**) digunakan dengan tepat.

7.2 No Circular Dependency

Status: ☒ PASSED

- Tidak ada dependensi melingkar antar kelas utama.

Visualisasi Dependency Flow:

Mahasiswa → KRS → Enrollment → Course. KRS → KRSValidationService (one-way).

7.3 Cardinality Verification

Status: ☒ PASSED

- Semua kardinalitas logis dan sesuai *business rules*.

8. KESIMPULAN DAN REKOMENDASI

8.1 Kesimpulan Utama

Desain sistem menunjukkan **arsitektur yang matang** dengan struktur *well-organized* dan *maintainable*. Desain telah mematuhi dan memverifikasi **5 prinsip SOLID** serta logika kardinalitas, menjadikannya **siap untuk diimplementasikan** dengan *confidence* tinggi.

8.2 Kekuatan (Strengths)

No	Aspek	Keterangan
1	Extensibility	Mudah diperluas untuk fitur baru
2	Maintainability	<i>Service layer</i> memudahkan <i>maintenance</i> logika bisnis
3	Testability	Struktur yang terpisah memudahkan <i>unit testing</i>

8.3 Area Perhatian (Considerations)

No	Area	Rekomendasi
1	Kompleksitas KRS	<i>Testing</i> komprehensif untuk semua skenario validasi
2	Grade Calculation	Dokumentasi detail algoritma perhitungan IPK

3	Concurrency	Pertimbangkan <i>locking mechanism</i> saat <i>KRS submission</i>
---	--------------------	---

8.4 Rekomendasi Implementasi

Fase 1: Core Infrastructure

- **Priority:** HIGH
- Implementasi hierarki **User** (Mahasiswa, Dosen, Admin).
- Sistem autentikasi dan otorisasi.
- Pembuatan skema *database* dasar.

Fase 2: Academic Core

- **Priority:** HIGH
- Implementasi **Course**, **Schedule**, **Semester**.
- Implementasi **KRS** dasar (tanpa validasi penuh).

Fase 3: Business Logic

- **Priority:** MEDIUM
- Implementasi **KRSValidationService** dengan semua aturan.
- Implementasi **GradeCalculationService**.
- Implementasi alur kerja **Enrollment**.

8.5 Rekomendasi Teknis

- **Technology Stack:** Spring Boot (Java) / Django (Python) / Laravel (PHP).
 - **Database:** PostgreSQL (untuk integritas data akademik).
 - **Caching:** Redis (untuk *session*, data yang sering diakses).
 - **Security:** Implementasi *Role-Based Access Control* (RBAC), *password hashing*.
-

9. LAMPIRAN

9.1 Glossary

Term	Definisi
IPK	Indeks Prestasi Kumulatif - <i>Cumulative GPA</i>
KRS	Kartu Rencana Studi - <i>Study Plan Card</i>
PMB	Penerimaan Mahasiswa Baru - <i>New Student Admission</i>
SKS	Satuan Kredit Semester - <i>Credit Unit</i>

9.2 Enumerations

PMBStatus

DRAFT, SUBMITTED, IN_REVIEW, APPROVED, REJECTED, ENROLLED

KRSStatus

DRAFT, SUBMITTED, APPROVED, REJECTED

EnrollmentStatus

ENROLLED, DROPPED, FAILED, PASSED

StudentStatus

ACTIVE, CUTI, GRADUATED, ON_LEAVE

DocumentType

TRANSKIP, KTP, IJAZAH, FOTO, SURAT_KETERANGAN_SEHAT

9.3 Design Patterns Identified

Pattern	Implementasi	Lokasi
Abstract Factory	Hierarki User	User, Mahasiswa, Dosen, Admin
Bridge Pattern	Relasi <i>Many-to-many</i>	Enrollment (<i>bridge</i> antara Mahasiswa & Course)
Service Layer	Pemisahan logika bisnis	KRSValidationService, GradeCalculationService
Strategy Pattern	(Potensial) Perhitungan Grade	GradeCalculationService
